

Product Review Search with Cassandra Vector DB



Estimated time needed: **30** minutes

Objectives

After completing this lab, you will be able to

- Generate vector embeddings for product reviews data using AI models
- Connect to the Cassandra database and create table with vector embeddings
- Write an endpoint using Express server that takes a sentence as input and returns the closest product based on the review

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands on labs for course and project related labs. It is an open-source IDE (Integrated Development Environment).

Important Notice about this lab environment

Please be aware that sessions for this lab environment are not persisted. Every time you connect to this lab, a new environment is created for you. Any data you may have saved in the earlier session would get lost. Plan to complete these labs in a single session, to avoid losing your data.

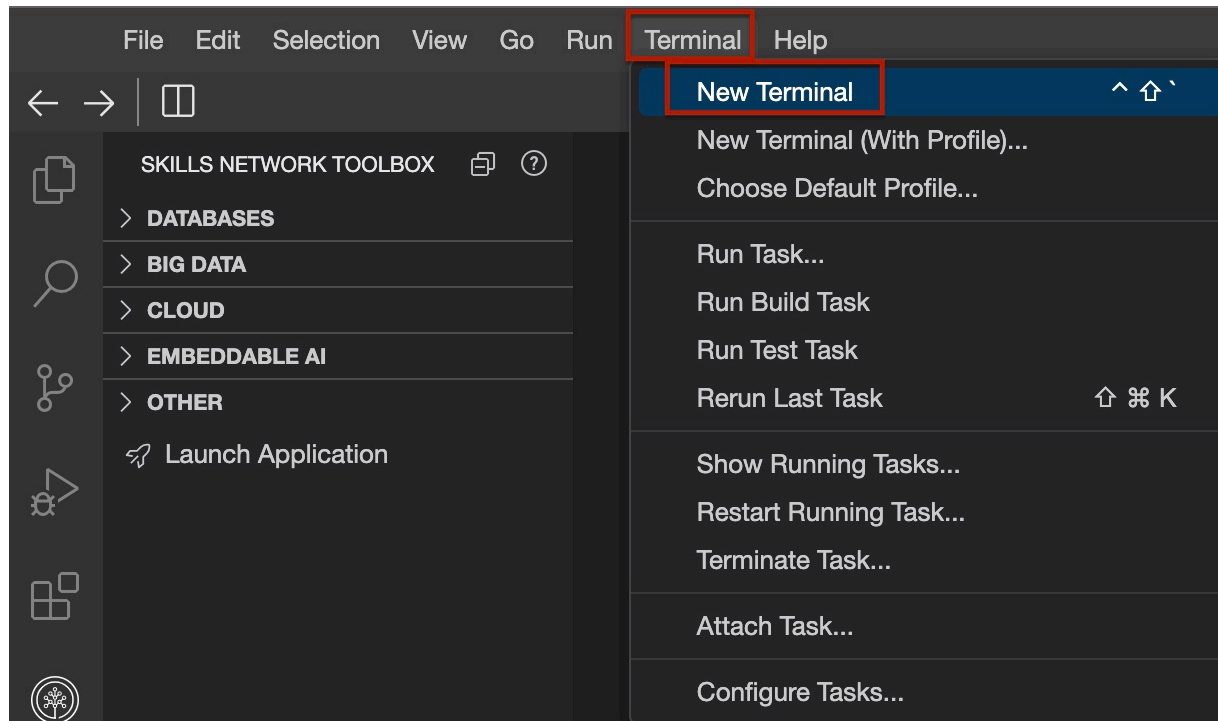
Task 1: Prerequisites

Start the Cassandra database

Cassandra database is provided in the lab environment. Follow the steps in the lab *Songs Discovery Service with Cassandra Vector DB* under the *Prerequisites* section to start the database. Note down the username and password once the database has started.

Set up the directory structure

1. Open a terminal in the lab environment using the *Terminal -> New Terminal* menu.



2. Next, you will create the home directory with the name `cassandra-product`.

```
mkdir cassandra-product && cd cassandra-product
```

Save the full path of the folder in an environment variable that you can refer to later.

```
export VECTOR_HOME=$(pwd)
```

You can use the `echo` command to see what's inside this variable.

```
echo $VECTOR_HOME
```

An example of the folder path is provided. Your folder structure might be different based on operating system and the way your user directory is set up.

```
echo $VECTOR_HOME  
/home/project/cassandra-product
```

Next, create a data sub directory using this variable.

```
mkdir $VECTOR_HOME/data
```

You should see a data directory inside project directory.

```
ls -ltr  
total 4  
drwxr-sr-x 2 theia users 4096 Apr 18 18:00 data
```

Note: Cassandra v5.0 offers dedicated vector search functionality, which simplifies vector storage, indexing, and searching processes. You will need to use all these features in the lab.

Here is a list of all similarity functions available in Cassandra:

- `cosine`: Measures the cosine of the angle between two vectors. A higher cosine value indicates greater similarity.
- `euclidean`: Measures the "straight-line" distance between two points (vectors) in multi-dimensional space. Lower distance indicates higher similarity.
- `dotproduct`: Calculates the sum of the product of corresponding components between two vectors. Higher dot product values indicate greater alignment.

[Learn more in the Cassandra documentation](#)

Well done! You have successfully set up the directory structure.

Seed Data

Before you start writing any code, let's look at the data file quickly. The seed data has been provided as a csv file containing an array of product reviews. Each review in the file has the following attributed:

- `id`: unique id of the review
- `brand`: brand of the product
- `categories`: categories the product belongs to
- `dateAdded`: date the review was added
- ...
- `reviews.text`: long form review
- `reviews.title`: short form review
- ...

Each row has 27 columns. We are only interested in the single column named `reviews.text`. The rest of the columns are left in place as in the real world, you will get files that need to be cleaned up before you can use them.

Sample Reviews

Review 1:

```
I have the Amazon Tap (which I purchased at Best Buy about a month ago) and love it so much I bought one for my son and his family. They love it, too!
```

Review 2:

```
First I would like to say that I am coming from using Google TV. Fire TV is a great device. The voice search is accurate at picking up what I say. My biggest problem with this device is also with the voice search. Coming from Google TV if you used the search feature i
```

Review 3:

```
This is fun but would not buy again. Not very informative. Keeps say "I do not understand your question. "
```

Notice, the reviews can be short or long.

Data Attribution

“[Amazon Product Reviews Dataset](#)” by [M Yasser H](#) is licensed under [CC0 1.0 DEED](#). Only 30 rows are available in the database for the purpose of this lab.

You can download the complete file using the following `wget` or `curl` command or simply opening this [URL](#) in your browser and saving the file locally.

Use of the following commands in the lab environment.

WGET

```
wget -O $VECTOR_HOME/data/products.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/6B04dP9cuhj6kfra5W3aSA/products.csv
```

CURL

```
curl -o $VECTOR_HOME/data/products.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/6B04dP9cuhj6kfra5W3aSA/products.csv
```

Task 1: Create Node.js project and install dependencies

You will initialize a Node project and install all the dependencies.

1. Ensure you are still in the \$VECTOR_HOME directory and Run `npm init -y` command.

```
npm init -y
```

Here is a sample output.

```
→ npm init -y
Wrote to /home/project/cassandra-product/package.json:
{
  "name": "cassandra-product",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

2. Install required packages including `@tensorflow-models/universal-sentence-encoder`, `@tensorflow/tfjs-node`, `dotenv`, `cassandra-driver`, `csv parser`, and `express`.

- `@tensorflow-models/universal-sentence-encoder`: The universal sentence encoder model
- `@tensorflow/tfjs-node`: JavaScript version of TensorFlow
- `dotenv`: Loads environment variables from a `.env` file into `process.env`
- `cassandra-driver`: Library to work with Cassandra DB

- csv-parser: To read and parse csv file
- express: Web framework for Node.js.

Hint - use the `npm install` command

► [Click here for the answer](#)

Task 2: Create common util functions

Create util.js file that has the common reusable code

1. Create a new file named util.js.

```
touch util.js
```

2. Open the util.js file in an editor and paste the following code in it.

Please read the code comments to understand what each part of the code does.

```
require('dotenv').config();
const cassandra = require('cassandra-driver');
const TimeUuid = cassandra.types.TimeUuid;
require("@tensorflow/tfjs-node");
const us_encoder = require("@tensorflow-models/universal-sentence-encoder");
let client;
let model;
// function to connect to the database and return a client.
async function connectToCassandra() {
  client = new cassandra.Client({
    contactPoints: [process.env.CASSANDRA_HOST],
    port: process.env.CASSANDRA_PORT,
    localDataCenter: 'datacenter1',
    authProvider: new cassandra.auth.PlainTextAuthProvider(process.env.CASSANDRA_USER, process.env.CASSANDRA_PASS)
  });
  await client.connect();
  console.log('Connected to Cassandra');
  return client;
}
// method that ensures there is a keyspace and table created in the database.
async function ensureKeyspaceAndTable() {
  console.log(`Ensuring KEYSPACE ${process.env.CASSANDRA_KEYSPACE} and TABLE ${process.env.CASSANDRA_TABLE} exists ...`);
  const keyspaceQuery = `CREATE KEYSPACE IF NOT EXISTS ${process.env.CASSANDRA_KEYSPACE} WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1}`;
  await client.execute(keyspaceQuery);
  const tableQuery = `CREATE TABLE IF NOT EXISTS ${process.env.CASSANDRA_KEYSPACE}.${process.env.CASSANDRA_TABLE} (
    id UUID PRIMARY KEY,
    asins text,
    brand text,
    categories text,
    colors text,
    dateAdded timestamp,
    dateUpdated timestamp,
    reviews_title text,
    reviews_userCity text,
    reviews_userProvince text,
    reviews_username text,
    embedding VECTOR<FLOAT, 512>
  )`;
  await client.execute(tableQuery);
  console.log(`KEYSPACE ${process.env.CASSANDRA_KEYSPACE} and TABLE ${process.env.CASSANDRA_TABLE} created`);
  // Ensure the vector index exists for efficient vector search
  const indexQuery = `CREATE INDEX IF NOT EXISTS ann_index ON ${process.env.CASSANDRA_KEYSPACE}.${process.env.CASSANDRA_TABLE}(embedding) USING 'sai';`;
  await client.execute(indexQuery);
  console.log('Index created');
}
// we cache the model so we don't have to fetch it every time
async function loadModel() {
  if (!model) {
    model = await us_encoder.load();
  }
}
async function vectorizeText(text) {
  await loadModel();
  const embeddings = await model.embed([text]);
  return embeddings.arraySync()[0]; // return the first embedding
}
async function closeCassandraConnection() {
  if (client) {
    await client.shutdown();
    console.log('Cassandra connection closed');
  }
}
```

```
}  
}  
// export all functions and variables so other files can use it.  
module.exports = {  
  connectToCassandra,  
  ensureKeyspaceAndTable,  
  vectorizeText,  
  closeCassandraConnection,  
  TimeUuid,  
  CASSANDRA_KEYSPACE: process.env.CASSANDRA_KEYSPACE,  
  CASSANDRA_TABLE: process.env.CASSANDRA_TABLE  
};
```

Refer to the **Songs Discovery Service with Cassandra Vector DB** lab to review the explanation for each and line of code.

Task 3: Load the vectors into Cassandra DB

Initial data

The product reviews are saved in the `product.json` file. The goal of this task is to read the `product.json` file into memory and then create vector embeddings for each `review.text` and store them in the database along with the rest of the columns.

Set up configuration

1. Create a file called `.env` in the `$VECTOR_HOME` directory.

```
touch .env
```

Populate the file with the following information:

```
CASSANDRA_HOST=localhost  
CASSANDRA_PORT=9042  
CASSANDRA_USER=  
CASSANDRA_PASS=  
CASSANDRA_KEYSPACE=vks  
CASSANDRA_TABLE=vectors
```

The `CASSANDRA_USER` and `CASSANDRA_PASS` should match the username and password provided to you when you started the database.

File to upload vectorized data into the collection

Create a new file named `loader.js`.

```
touch loader.js
```

The loader.js file will contain the method that loads the initial news articles to the Cassandra database.

```
const fs = require('fs');
const csv = require('csv-parser');
const { connectToCassandra, ensureKeyspaceAndTable, vectorizeText, closeCassandraConnection, TimeUuid, CASSANDRA_KEYSPACE, CASSANDRA_TABLE } = require('./util');
async function loadVectorizedData() {
  const client = await connectToCassandra();
  await ensureKeyspaceAndTable();
  const operations = [];
  const dataStream = fs.createReadStream('./data/products.csv')
    .pipe(csv())
    .on('data', (row) => {
      dataStream.pause();
      const operation = async () => {
        try {
          const title = row['reviews.title'] ? row['reviews.title'].replace(/'/g, '') : '';
          console.log(`Read title: ${title}`);
          const embedding = await vectorizeText(title);
          const embeddingStr = embedding.map(value => value.toFixed(6)).join(',');
          const query = `INSERT INTO ${CASSANDRA_KEYSPACE}.${CASSANDRA_TABLE} (
            id, asins, brand, categories, colors, dateAdded, dateUpdated, reviews_title, reviews_userCity,
            reviews_userProvince, reviews_username, embedding)
          VALUES (
            ${TimeUuid.now()},
            ${row['asins']},
            ${row['brand']},
            ${row['categories']},
            ${row['colors']},
            ${row['dateAdded']},
            ${row['dateUpdated']},
            ${title},
            ${row['reviews.userCity']},
            ${row['reviews.userProvince']},
            ${row['reviews.username']},
            [${embeddingStr}]
          );`;
          await client.execute(query);
        } catch (error) {
          console.error('Error inserting data:', error);
        } finally {
          dataStream.resume();
        }
      };
      operations.push(operation());
    })
    .on('end', async () => {
      await Promise.all(operations);
      await closeCassandraConnection();
      console.log('Finished loading data');
    });
  dataStream.read();
}
loadVectorizedData();
```

The code was explained in detail in the [Songs Discovery Service with Cassandra Vector DB](#) lab. Revisit the lab if anything is unclear.

Run the code

1. Install the packages

Install all the packages using the npm install command.

```
npm install
```

2. Run the file

Run the file using the node command.

```
node loader.js
```

You should see the product reviews load one by one. Your output might look different than what is shown here.

```
Connected to Cassandra
Ensuring KEYSPACE vks and TABLE vectors exists ...
KEYSPACE vks and TABLE vectors created
Index created
Read title: Great Product!
Read title: Great device with some issues, but overall great device.
Read title: A little fun but not very useful
Read title: Great Sound and Versatility
Read title: kindle fire HDX
Read title: Ok but the echo is better
Read title: Apri 28th update - Good upgrade from last year model-Beautiful Display, Great Device-Fell short on battery life 18,548 people found this helpful. Was this review helpful to you Yes No
Read title: Great
Read title: Amazon Fire is a great choice
Read title: Great case for a great Tablet!
Read title: Facts to Consider when Buying This Kindle Instead of Others
Read title: This box is a GAME CHANGER for on demand content! Amazon knocked it out of the park. Refreshed UPDATED with Tips! and HBO GO! 26,755 people found this helpful. Was this review helpful to you Yes No
Read title: Gift product
Read title: Amazon fire tv remote sucks!
Read title: Great Amazon tap
Read title: so far we are very happy - especially kids as they have all their favorite
Read title: I also own a Echo and if you own a Echo you will love the Tap
Read title: its ok
Read title: Love Alexa
Read title: Great
Read title: very good product
Read title: HD Fire 6 is great for the price! 3,542 people found this helpful. Was this review helpful to y
theia@theiadocker-captainfedoi:/home/project/cassandra-product$
```

If no errors are displayed, it means the product reviews were vectorized and loaded into the collection.

Task 4: Create service endpoint

This task is left for you to do on your own! Create a server.js file as you did in the lab Songs Discovery Service with Cassandra Vector DB under 'Task 6'. You will implement an endpoint that will return the title closest to the user query.

```
curl -X POST http://localhost:3000/search \
-H "Content-Type: application/json" \
-d '{"text": "was working well, but lately does not work at all"}' | jq .
```

Congratulations! You successfully created vector embeddings for song titles, created an index on this vector and finally created an endpoint that can be used to retrieve similar product reviews to user input!

Authors

UL